LAPORAN PRAKTIKUM BAB VI
KELOMPOK 2



Disusun untuk Memenuhi Tugas
Mata Kuliah:
Workshop DevOps

Dosen Pengampu :
Dr. Ferry Astika Saputra, ST, M.Sc

Oleh:
Ibram Maulana Akhsanul Qasasi NRP. 3126640014
Yusuf Rahmadhani Asy'ari NRP. 3126640042
Vernanda Mulia Hamonangan Manurung NRP. 3126640044

PROGRAM STUDI D4 LANJUT JENJANG TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

BAB VI

CENTRALIZED LOGGING DENGAN FLUENT BIT DAN POSTGRESQL

6.1 Tahap Praktikum

6.1.1 Persiapan Konfigurasi dan Pembuatan File

Kotak Script 1 — File .env.example

Dilakukan pembuatan file .env berdasarkan .env.example menggunakan perintah cp .env.example .env. File .env.example berfungsi sebagai contoh konfigurasi variabel lingkungan yang dibutuhkan oleh PostgreSQL dan Fluent Bit, seperti nama database, pengguna database, dan password. Perintah chmod 600 .env digunakan untuk membatasi hak akses file agar hanya pemiliknya yang dapat membaca dan mengubah file tersebut. Hal ini penting karena file .env menyimpan informasi sensitif yang digunakan oleh layanan. Pemeriksaan menggunakan stat -c '%a %n' .env menunjukkan izin 600, yang berarti akses terhadap file telah dibatasi. Dalam penerapan sebenarnya, password harus menggunakan nilai yang kuat dan tidak boleh membagikan file .env ke repositori publik.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cp .env.example .env
chmod 600 .env
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat .env.example
POSTGRES_DB=labdb
POSTGRES_USER=labadmin
POSTGRES_PASSWORD=labadminpass123
FLUENTBIT_DB_USER=fluentbit_user
FLUENTBIT_DB_PASSWORD=fluentbitpass123
ubuntu-2@ubn-2:~/docker-lab/bab-6$ ls -la^C
ubuntu-2@ubn-2:~/docker-lab/bab-6$ ls -la
total 32
drwxrwxr-x 6 ubuntu-2 ubuntu-2 4096 Okt 20 05:08 .
drwxrwxr-x 9 ubuntu-2 ubuntu-2 4096 Okt 20 05:07 ..
-rw------- 1 ubuntu-2 ubuntu-2  147 Okt 20 05:08 .env
-rw-rw-r-- 1 ubuntu-2 ubuntu-2  147 Okt 20 05:07 .env.example
drwxrwxr-x 2 ubuntu-2 ubuntu-2 4096 Okt 20 05:07 fluent-bit
drwxrwxr-x 2 ubuntu-2 ubuntu-2 4096 Okt 20 05:07 init
drwxrwxr-x 2 ubuntu-2 ubuntu-2 4096 Okt 20 05:07 queries
drwxrwxr-x 2 ubuntu-2 ubuntu-2 4096 Okt 20 05:07 scripts
ubuntu-2@ubn-2:~/docker-lab/bab-6$ stat -c '%a %n' .env
600 .env
ubuntu-2@ubn-2:~/docker-lab/bab-6$
```

Kotak Script 2 — File .gitignore

Diperlihatkan isi file .gitignore yang digunakan untuk menentukan file atau direktori yang tidak perlu dimasukkan ke dalam repositori Git. Konfigurasi .env mencegah file yang berisi konfigurasi sensitif ikut dilacak oleh Git, sedangkan *.log digunakan untuk mengabaikan file yang memiliki ekstensi log dan reports/ untuk mengabaikan direktori laporan. Pengaturan ini membantu mengurangi risiko kebocoran informasi sensitif dan menjaga repositori tetap berisi berkas yang diperlukan untuk pengembangan. Namun, .gitignore tidak menghapus file yang sebelumnya sudah terlanjur dilacak oleh Git, sehingga file sensitif yang sudah terunggah perlu ditangani secara terpisah.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat .gitignore
.env
*.log
reports/
```

Kotak Script 3 — File compose.yaml

Dibuat konfigurasi Docker Compose untuk menjalankan PostgreSQL, Fluent Bit, dan tiga container pembangkit log, yaitu generator-auth, generator-api, dan generator-worker. PostgreSQL menggunakan image postgres:16-alpine, sedangkan Fluent Bit menggunakan image fluent/fluent-bit:4.2.8. Variabel lingkungan digunakan agar konfigurasi database dan password dapat diambil dari file .env tanpa menuliskannya langsung sebagai nilai tetap di dalam konfigurasi Compose.

Bagian volumes pada layanan PostgreSQL menggunakan named volume pg-data untuk menyimpan data database agar tetap tersedia meskipun container dibuat ulang. Direktori ./init dipasang ke /docker-entrypoint-initdb.d dengan opsi ro, yang berarti berkas inisialisasi hanya dapat dibaca dari dalam container. Opsi ini membantu mencegah perubahan terhadap berkas inisialisasi melalui container. Konfigurasi healthcheck dengan perintah pg_isready digunakan untuk memeriksa apakah PostgreSQL sudah siap menerima koneksi. Sementara

itu, depends_on dengan kondisi service_healthy membuat Fluent Bit menunggu sampai layanan PostgreSQL dinyatakan sehat sebelum dijalankan.

Ketiga layanan generator menggunakan image Alpine dan perintah shell untuk menghasilkan log dengan isi yang berbeda. Generator autentikasi menghasilkan event login, generator API menghasilkan event pengujian latensi, dan generator worker menghasilkan event percobaan ulang. Setiap generator mengirimkan log melalui Docker logging driver fluentd menuju alamat Fluent Bit pada port 24224. Opsi fluentd-async: "true" memungkinkan pengiriman log secara asinkron sehingga proses aplikasi tidak harus selalu menunggu penerima log siap pada saat yang sama. Tag docker.{{.Name}} digunakan untuk membedakan sumber log berdasarkan nama container. Selain itu, port monitoring 2020 dan port penerimaan log 24224 hanya dipublikasikan pada alamat 127.0.0.1, sehingga tidak langsung dibuka ke seluruh antarmuka jaringan host.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat compose.yaml
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
      FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - logging-net
    healthcheck:
      test:
        - CMD-SHELL
        - "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"
      interval: 5s
      timeout: 5s
      retries: 12
      start_period: 10s
    restart: unless-stopped

  fluent-bit:
    image: fluent/fluent-bit:4.2.8
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
      FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    volumes:
      - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
      - flb-storage:/var/log/flb-storage
    ports:
      - "127.0.0.1:24224:24224"
      - "127.0.0.1:2020:2020"
    networks:
      - logging-net
    depends_on:
      postgres-db:
        condition: service_healthy
    security_opt:
      - no-new-privileges:true
    restart: unless-stopped

  generator-auth:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"auth","severity":"INFO","event":"login_test","sequence":%s}\n' "$$i"
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit

  generator-api:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"api","severity":"WARN","event":"latency_test","duration_ms":%s}\n' "$$((i * 10))"
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit

  generator-worker:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"worker","severity":"ERROR","event":"retry_test","attempt":%s}\n' "$$i"
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit
```

6.1.2 Pembuatan Database dan Konfigurasi Pengumpulan Log

Kotak Script 4 — File init/01-init-logging.sh

Script inisialisasi PostgreSQL yang bertugas menyiapkan akun pengguna, schema, tabel, indeks, view, dan hak akses yang dibutuhkan Fluent Bit. Baris set -Eeuo pipefail membuat script lebih ketat dalam menangani kesalahan. Opsi -e membantu menghentikan script ketika perintah gagal, -u membuat penggunaan variabel yang belum ditentukan dianggap sebagai kesalahan, sedangkan pipefail membuat kegagalan perintah dalam pipeline tidak mudah terabaikan. Pemeriksaan variabel seperti POSTGRES_USER, POSTGRES_DB, FLUENTBIT_DB_USER, dan FLUENTBIT_DB_PASSWORD memastikan nilai yang dibutuhkan tersedia sebelum proses inisialisasi dilakukan.

Perintah psql dijalankan dengan ON_ERROR_STOP=1 agar kesalahan SQL dapat menghentikan proses inisialisasi. Script menggunakan CREATE ROLE untuk membuat pengguna khusus Fluent Bit apabila pengguna tersebut belum tersedia. Penggunaan format() bersama %I untuk identifier SQL dan %L untuk literal SQL membantu menghasilkan perintah SQL dengan penanganan nilai yang sesuai. Perintah \gexec pada psql menjalankan perintah SQL yang dihasilkan oleh query sebelumnya. Dengan cara ini, pembuatan role dapat dilakukan secara bersyarat sehingga script lebih aman ketika dijalankan kembali.

Selanjutnya, CREATE SCHEMA IF NOT EXISTS logs membuat schema logs jika belum tersedia, sedangkan CREATE TABLE IF NOT EXISTS logs.fluentbit menyiapkan tabel untuk menyimpan tag, waktu penerimaan, dan data log dalam tipe JSONB. Indeks pada kolom waktu dan tag membantu mempercepat pencarian berdasarkan waktu maupun sumber log. Indeks GIN pada kolom data berguna untuk mendukung pencarian terhadap isi JSON. Script juga membuat view logs.recent_logs yang menampilkan log terbaru beserta nama container, sumber log, dan cuplikan pesan. Terakhir, perintah GRANT memberikan hak akses yang diperlukan kepada pengguna Fluent Bit, yaitu penggunaan schema serta hak membaca dan memasukkan data ke tabel. Pembatasan hak akses ini lebih baik daripada menggunakan akun administrator database untuk proses pengiriman log sehari-hari.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat init/01-init-logging.sh
#!/usr/bin/env bash
set -Eeuo pipefail

: "${POSTGRES_USER:?POSTGRES_USER belum diatur}"
: "${POSTGRES_DB:?POSTGRES_DB belum diatur}"
: "${FLUENTBIT_DB_USER:?FLUENTBIT_DB_USER belum diatur}"
: "${FLUENTBIT_DB_PASSWORD:?FLUENTBIT_DB_PASSWORD belum diatur}"

psql \
  --set=ON_ERROR_STOP=1 \
  --username "$POSTGRES_USER" \
  --dbname "$POSTGRES_DB" \
  --set=fb_user="$FLUENTBIT_DB_USER" \
  --set=fb_password="$FLUENTBIT_DB_PASSWORD" <<'SQL'
SELECT format(
    'CREATE ROLE %I LOGIN PASSWORD %L',
    :'fb_user',
    :'fb_password'
)
WHERE NOT EXISTS (
    SELECT 1 FROM pg_roles WHERE rolname = :'fb_user'
)
\gexec

CREATE SCHEMA IF NOT EXISTS logs;

CREATE TABLE IF NOT EXISTS logs.fluentbit (
    tag  TEXT NOT NULL,
    time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    data JSONB NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_fluentbit_time
    ON logs.fluentbit (time DESC);

CREATE INDEX IF NOT EXISTS idx_fluentbit_tag
    ON logs.fluentbit (tag);

CREATE INDEX IF NOT EXISTS idx_fluentbit_data_gin
    ON logs.fluentbit USING GIN (data);

CREATE OR REPLACE VIEW logs.recent_logs AS
SELECT
    time,
    tag,
    replace(data ->> 'container_name', '/', '') AS container_name,
    data ->> 'source' AS source,
    left(data ->> 'log', 200) AS log_preview
FROM logs.fluentbit
ORDER BY time DESC
LIMIT 100;

SELECT format(
    'ALTER ROLE %I IN DATABASE %I SET search_path TO logs, public',
    :'fb_user',
    current_database()
)
\gexec

SELECT format('GRANT USAGE, CREATE ON SCHEMA logs TO %I', :'fb_user')
\gexec

SELECT format(
    'GRANT SELECT, INSERT ON TABLE logs.fluentbit TO %I',
    :'fb_user'
)
\gexec

SELECT format('GRANT SELECT ON TABLE logs.recent_logs TO %I', :'fb_user')
\gexec
SQL
```

Kotak Script 5 — File fluent-bit/fluent-bit.conf

Dilakukan konfigurasi Fluent Bit sebagai penerima dan penerus log. Bagian [SERVICE] mengatur perilaku umum Fluent Bit, termasuk tingkat pencatatan pesan internal melalui Log_Level info, penggunaan penyimpanan buffer melalui storage.path, dan layanan HTTP untuk monitoring. Pengaturan storage.sync normal serta storage.checksum on membantu mengatur penyimpanan buffer dan pemeriksaan integritas data buffer. Batas storage.backlog.mem_limit 10M mengatur batas memori untuk backlog yang sedang diproses dari penyimpanan. Pengaturan ini penting agar pengelolaan log tidak menggunakan memori secara berlebihan.

Bagian [INPUT] menggunakan plugin forward untuk menerima log dari Docker logging driver melalui port 24224. Pengaturan Mem_Buf_Limit 10M membatasi buffer dalam memori, sedangkan storage.type filesystem memungkinkan buffer menggunakan penyimpanan file ketika diperlukan. Mekanisme ini membantu menangani kondisi ketika tujuan pengiriman log mengalami gangguan sementara. Bagian [OUTPUT] pertama menggunakan plugin pgsql untuk mengirimkan log ke PostgreSQL dengan mengambil nama database dan kredensial dari variabel lingkungan. Pengaturan Match * berarti output tersebut menerima semua tag yang cocok. Sementara itu, output kedua menggunakan plugin stdout dengan format json_lines agar data log juga dapat ditampilkan dalam keluaran standar untuk membantu pemeriksaan dan debugging. Dengan konfigurasi ini, log tidak hanya diterima, tetapi juga diarahkan ke database untuk disimpan dan dianalisis.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ nano fluent-bit/fluent-bit.conf
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat fluent-bit/fluent-bit.conf
[SERVICE]
    Flush                     1
    Daemon                    Off
    Log_Level                 info
    storage.path              /var/log/flb-storage
    storage.sync              normal
    storage.checksum          on
    storage.backlog.mem_limit 10M
    HTTP_Server               On
    HTTP_Listen               0.0.0.0
    HTTP_Port                 2020

[INPUT]
    Name          forward
    Listen        0.0.0.0
    Port          24224
    Mem_Buf_Limit 10M
    storage.type  filesystem

[OUTPUT]
    Name                      pgsql
    Match                     *
    Host                      postgres-db
    Port                      5432
    User                      ${FLUENTBIT_DB_USER}
    Password                  ${FLUENTBIT_DB_PASSWORD}
    Database                  ${POSTGRES_DB}
    Table                     fluentbit
    storage.total_limit_size 256M

[OUTPUT]
    Name   stdout
    Match  *
    Format json_lines
```

6.1.3 Pembuatan Query dan Script Pengujian

Kotak Script 6 — File queries/analysis.sql

Dibuat kumpulan query SQL untuk menganalisis data log yang telah disimpan dalam tabel logs.fluentbit. Perintah \pset pager off digunakan untuk menonaktifkan tampilan pager pada psql, sedangkan \timing on menampilkan waktu eksekusi query. Query pertama menghitung jumlah seluruh event menggunakan COUNT(*) serta mengambil waktu event paling awal dan paling akhir melalui MIN(time) dan MAX(time). Informasi tersebut membantu mengetahui jumlah data yang sudah diterima dan rentang waktu pencatatannya.

Query kedua menggunakan GROUP BY tag untuk menghitung jumlah log dari setiap sumber. Hasilnya dapat digunakan untuk memastikan bahwa log dari generator autentikasi, API, dan worker masuk ke database. Query ketiga mengambil metadata dari kolom JSONB menggunakan operator ->>, yang digunakan untuk mengambil nilai sebuah properti JSON sebagai teks. Fungsi replace() menghapus karakter garis miring dari nama container, sedangkan left() membatasi panjang cuplikan pesan agar hasil query lebih mudah dibaca. Query keempat mencari pesan yang memuat informasi tingkat kesalahan ERROR menggunakan ILIKE, yaitu pencarian teks tanpa membedakan huruf besar dan kecil. Query kelima menggunakan date_trunc('minute', time) untuk mengelompokkan event berdasarkan menit sehingga laju penerimaan log dapat diamati. Terdapat pula contoh query retensi yang dinonaktifkan dengan tanda komentar. Query penghapusan tersebut tidak dijalankan secara otomatis, sehingga penghapusan data lama perlu ditinjau terlebih dahulu sebelum diterapkan.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ nano queries/analysis.sql
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat queries/analysis.sql
\pset pager off
\timing on

-- 1. Jumlah event dan rentang waktu.
SELECT
    count(*) AS total_event,
    min(time) AS event_pertama,
    max(time) AS event_terakhir
FROM logs.fluentbit;

-- 2. Distribusi berdasarkan tag Docker.
SELECT
    tag,
    count(*) AS total
FROM logs.fluentbit
GROUP BY tag
ORDER BY total DESC, tag;

-- 3. Metadata terstruktur dari Docker logging driver.
SELECT
    time,
    tag,
    replace(data ->> 'container_name', '/', '') AS container_name,
    data ->> 'source' AS source,
    left(data ->> 'log', 160) AS log_preview
FROM logs.fluentbit
ORDER BY time DESC
LIMIT 15;

-- 4. Pencarian severity pada JSON yang masih berada di field log.
SELECT
    time,
    tag,
    data ->> 'log' AS log_message
FROM logs.fluentbit
WHERE data ->> 'log' ILIKE '%"severity":"ERROR"%'
ORDER BY time DESC;

-- 5. Ringkasan per menit untuk mengamati laju ingest.
SELECT
    date_trunc('minute', time) AS minute_bucket,
    count(*) AS total
FROM logs.fluentbit
GROUP BY minute_bucket
ORDER BY minute_bucket DESC
LIMIT 10;

-- Contoh retensi; tinjau dahulu sebelum menghapus data.
-- DELETE FROM logs.fluentbit
-- WHERE time < CURRENT_TIMESTAMP - INTERVAL '7 days';
```

Kotak Script 7 — File scripts/generate-logs.sh

Dibuat script untuk menjalankan layanan yang dibutuhkan dan menghasilkan log pengujian secara otomatis. Baris set -Eeuo pipefail digunakan untuk memperketat penanganan kesalahan, sedangkan cd "$(dirname "$0")/.." memastikan script berpindah ke direktori utama proyek berdasarkan lokasi file script. Cara ini membuat script tidak bergantung pada direktori kerja saat perintah dijalankan. Pemeriksaan if [[! -f .env]] memastikan file konfigurasi tersedia sebelum layanan dijalankan. Jika file tidak ditemukan, script menampilkan pesan kegagalan dan menghentikan proses menggunakan exit 1.

Selanjutnya, perintah docker compose up -d postgres-db fluent-bit menjalankan PostgreSQL dan Fluent Bit di latar belakang. Script kemudian melakukan pemeriksaan berulang terhadap endpoint kesehatan Fluent Bit melalui curl -fsS ke alamat http://127.0.0.1:2020/api/v1/health. Opsi -f membantu mendeteksi respons HTTP yang menunjukkan kegagalan, -s menonaktifkan keluaran progres, dan -S tetap menampilkan pesan kesalahan. Jika endpoint belum siap setelah jumlah percobaan yang ditentukan, script menampilkan pesan gagal dan berhenti. Setelah Fluent Bit siap, ketiga generator dijalankan ulang menggunakan --force-recreate agar proses pengujian dimulai dari container baru. Script ini mempermudah pelaksanaan praktikum karena pemeriksaan kesiapan layanan dilakukan sebelum generator mulai mengirimkan log.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ nano scripts/generate-logs.sh
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat scripts/generate-logs.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

docker compose up -d postgres-db fluent-bit

for attempt in {1..30}; do
    if curl -fsS http://127.0.0.1:2020/api/v1/health >/dev/null; then
        break
    fi

    if [[ "$attempt" -eq 30 ]]; then
        echo "[FAIL] Endpoint kesehatan Fluent Bit belum siap." >&2
        exit 1
    fi
    sleep 2
done

docker compose up -d --force-recreate \
    generator-auth \
    generator-api \
    generator-worker

echo "[PASS] Tiga generator telah dijalankan. Tunggu sekitar 25 detik."
echo "[INFO] Pantau dengan: docker compose ps"
```

Kotak Script 8 — File scripts/test-buffering.sh

Dibuat script untuk menguji apakah Fluent Bit dapat mempertahankan log ketika PostgreSQL berhenti sementara dan melanjutkan pengiriman setelah database kembali aktif. Script terlebih dahulu memeriksa keberadaan .env, kemudian menggunakan set -a dan source .env agar variabel dari file tersebut diekspor ke lingkungan shell. Fungsi count_rows() menjalankan query SELECT count(*) FROM logs.fluentbit melalui docker compose exec -T postgres-db psql. Opsi -T menonaktifkan alokasi pseudo-terminal sehingga keluaran perintah lebih sesuai untuk diproses oleh script.

Jumlah data awal disimpan dalam variabel before. Setelah itu, PostgreSQL dihentikan sementara menggunakan docker compose stop postgres-db, sedangkan ketiga generator dijalankan kembali. Selama database berhenti, Fluent Bit diharapkan menampung log yang masuk menggunakan mekanisme buffering. Setelah waktu pengujian selesai, PostgreSQL dijalankan kembali dan script memeriksa kesiapan database menggunakan pg_isready. Pemeriksaan berulang dilakukan agar script tidak langsung menganggap database gagal hanya karena proses pemulihan membutuhkan waktu.

Terakhir, script menghitung jumlah data kembali dan membandingkannya dengan jumlah awal. Jika jumlah data meningkat, script menampilkan [PASS] beserta jumlah kenaikan record dan mengakhiri proses dengan kode keluar 0. Jika tidak ada penambahan record setelah batas percobaan, script menampilkan [FAIL] dan petunjuk untuk memeriksa log Fluent Bit. Mekanisme ini penting karena pengujian tidak hanya memeriksa apakah layanan berjalan, tetapi juga apakah log yang dihasilkan selama gangguan dapat diteruskan setelah database pulih.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ nano scripts/test-buffering.sh
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat scripts/test-buffering.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

count_rows() {
    docker compose exec -T postgres-db \
        psql \
        --username "$POSTGRES_USER" \
        --dbname "$POSTGRES_DB" \
        --tuples-only \
        --no-align \
        --command 'SELECT count(*) FROM logs.fluentbit;'
}

before="$(count_rows)"
echo "[INFO] Jumlah awal: $before"

docker compose stop postgres-db

docker compose up -d --force-recreate \
    generator-auth \
    generator-api \
    generator-worker

echo "[INFO] PostgreSQL berhenti; generator berjalan selama 25 detik."
sleep 25

docker compose start postgres-db

for attempt in {1..30}; do
    if docker compose exec -T postgres-db \
        pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" >/dev/null 2>&1; then
        break
    fi

    if [[ "$attempt" -eq 30 ]]; then
        echo "[FAIL] PostgreSQL tidak kembali sehat." >&2
        exit 1
    fi
    sleep 2
done

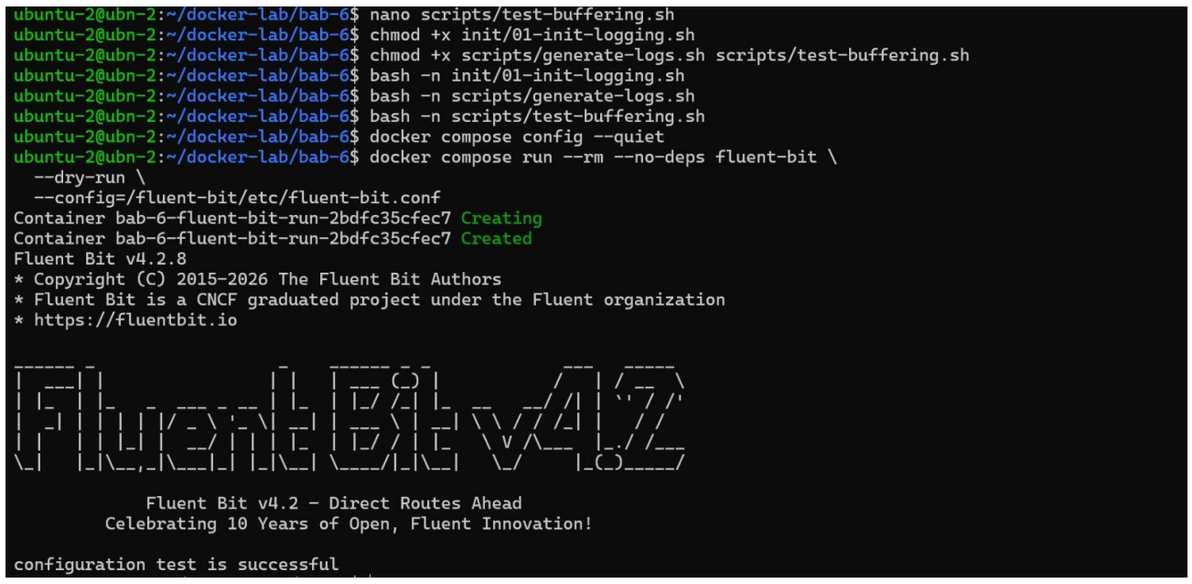
for attempt in {1..60}; do
    after="$(count_rows)"
    if (( after > before )); then
        echo "[PASS] Jumlah sesudah pemulihan: $after"
        echo "[INFO] Kenaikan record: $((after - before))"
        exit 0
    fi
    sleep 2
done

echo "[FAIL] Tidak ada record baru setelah PostgreSQL pulih." >&2
echo "[INFO] Periksa: docker compose logs --tail 200 fluent-bit" >&2
exit 1
```

6.1.4 Pelaksanaan dan Pemeriksaan Hasil Praktikum

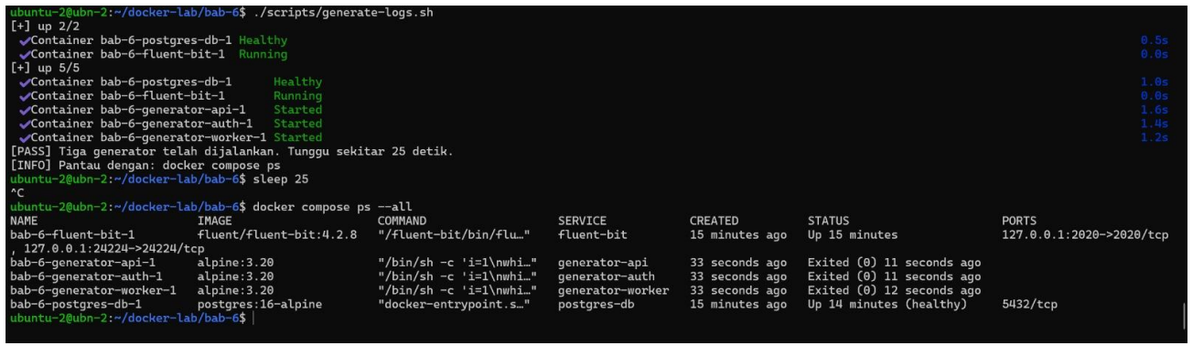
Kotak Perintah B — Memberi Izin Eksekusi dan Memvalidasi Berkas

Dilakukan persiapan script dan validasi konfigurasi sebelum pipeline dijalankan. Script shell perlu memiliki izin eksekusi agar dapat dipanggil sebagai program, misalnya melalui chmod +x scripts/generate-logs.sh dan chmod +x scripts/test-buffering.sh. Setelah itu, konfigurasi Docker Compose diperiksa menggunakan docker compose config --quiet. Perintah tersebut memvalidasi struktur dan sintaks konfigurasi Compose tanpa menampilkan seluruh konfigurasi yang sudah diproses. Jika perintah selesai tanpa kesalahan, konfigurasi dapat dilanjutkan ke tahap menjalankan layanan. Validasi ini penting karena kesalahan penulisan YAML, nama layanan, atau referensi variabel dapat menyebabkan layanan gagal dibuat.



Kotak Perintah C — Menjalankan Pipeline dan Menghasilkan Log

Pipeline dijalankan dengan memanggil script generate-logs.sh. Script akan menjalankan PostgreSQL dan Fluent Bit terlebih dahulu, memeriksa endpoint kesehatan, lalu memulai ketiga generator log. Setiap generator menghasilkan event dengan jenis yang berbeda, sehingga data yang masuk ke sistem dapat dibedakan berdasarkan tag dan isi pesannya. Pesan [PASS] menunjukkan bahwa script telah melewati pemeriksaan kesiapan Fluent Bit dan menjalankan generator. Setelah proses tersebut, log dibiarkan berjalan selama waktu yang ditentukan agar cukup banyak record terkumpul untuk dianalisis. Tahap ini menjadi bukti bahwa proses pengumpulan log dilakukan secara otomatis, bukan dengan memasukkan data secara manual ke database.



Kotak Perintah D — Memeriksa Monitoring Fluent Bit

Dilakukan pemeriksaan terhadap layanan Fluent Bit melalui endpoint monitoring HTTP pada port 2020. Endpoint kesehatan digunakan untuk memastikan bahwa layanan merespons permintaan pemeriksaan. Pemeriksaan ini membantu membedakan kondisi ketika container sekadar berstatus berjalan dengan kondisi ketika layanan di dalam container sudah siap merespons. Port monitoring dipublikasikan pada alamat loopback 127.0.0.1, sehingga pemeriksaan dilakukan dari host lokal dan tidak langsung membuka endpoint tersebut ke seluruh jaringan. Hasil pemeriksaan yang berhasil menunjukkan bahwa endpoint dapat diakses, tetapi tidak dengan sendirinya membuktikan bahwa seluruh log telah tersimpan di PostgreSQL. Oleh karena itu, hasil monitoring tetap perlu dilengkapi dengan pemeriksaan data pada database.



```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ set -a
source .env
set +a

docker compose exec -T postgres-db \
  psql \
  --username "$POSTGRES_USER" \
  --dbname "$POSTGRES_DB" \
  --file /dev/stdin \
  < queries/analysis.sql
```

```
Pager usage is off.
Timing is on.
 total_event |     event_pertama     |    event_terakhir
-------------+-----------------------+----------------------
         180 | 2026-10-20 05:36:59 | 2026-10-20 05:52:30
(1 row)
```

```
 total_event |     event_pertama     |    event_terakhir
-------------+-----------------------+----------------------
         180 | 2026-10-20 05:36:59 | 2026-10-20 05:52:30
(1 row)

Time: 1.884 ms
             tag               | total
-------------------------------+-------
 docker.bab-6-generator-api-1  |    60
 docker.bab-6-generator-auth-1 |    60
 docker.bab-6-generator-worker-1 |    60
(3 rows)

Time: 1.815 ms
        time         |              tag               |      container_name      | source |                                                                log_preview
---------------------+--------------------------------+--------------------------+--------+-------------------------------------------------------------------------------------------
 2026-10-20 05:52:30 | docker.bab-6-generator-worker-1 | bab-6-generator-worker-1 | stdout | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":20}
 2026-10-20 05:52:30 | docker.bab-6-generator-auth-1   | bab-6-generator-auth-1   | stdout | {"service":"auth","severity":"INFO","event":"login_test","sequence":20}
 2026-10-20 05:52:30 | docker.bab-6-generator-api-1    | bab-6-generator-api-1    | stdout | {"service":"api","severity":"WARN","event":"latency_test","duration_ms":200}
 2026-10-20 05:52:29 | docker.bab-6-generator-worker-1 | bab-6-generator-worker-1 | stdout | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":19}
 2026-10-20 05:52:29 | docker.bab-6-generator-auth-1   | bab-6-generator-auth-1   | stdout | {"service":"auth","severity":"INFO","event":"login_test","sequence":19}
 2026-10-20 05:52:29 | docker.bab-6-generator-api-1    | bab-6-generator-api-1    | stdout | {"service":"api","severity":"WARN","event":"latency_test","duration_ms":190}
 2026-10-20 05:52:28 | docker.bab-6-generator-worker-1 | bab-6-generator-worker-1 | stdout | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":18}
 2026-10-20 05:52:28 | docker.bab-6-generator-auth-1   | bab-6-generator-auth-1   | stdout | {"service":"auth","severity":"INFO","event":"login_test","sequence":18}
 2026-10-20 05:52:28 | docker.bab-6-generator-api-1    | bab-6-generator-api-1    | stdout | {"service":"api","severity":"WARN","event":"latency_test","duration_ms":180}
 2026-10-20 05:52:27 | docker.bab-6-generator-worker-1 | bab-6-generator-worker-1 | stdout | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":17}
 2026-10-20 05:52:27 | docker.bab-6-generator-auth-1   | bab-6-generator-auth-1   | stdout | {"service":"auth","severity":"INFO","event":"login_test","sequence":17}
 2026-10-20 05:52:27 | docker.bab-6-generator-api-1    | bab-6-generator-api-1    | stdout | {"service":"api","severity":"WARN","event":"latency_test","duration_ms":170}
 2026-10-20 05:52:26 | docker.bab-6-generator-worker-1 | bab-6-generator-worker-1 | stdout | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":16}
 2026-10-20 05:52:26 | docker.bab-6-generator-auth-1   | bab-6-generator-auth-1   | stdout | {"service":"auth","severity":"INFO","event":"login_test","sequence":16}
 2026-10-20 05:52:26 | docker.bab-6-generator-api-1    | bab-6-generator-api-1    | stdout | {"service":"api","severity":"WARN","event":"latency_test","duration_ms":160}
(15 rows)

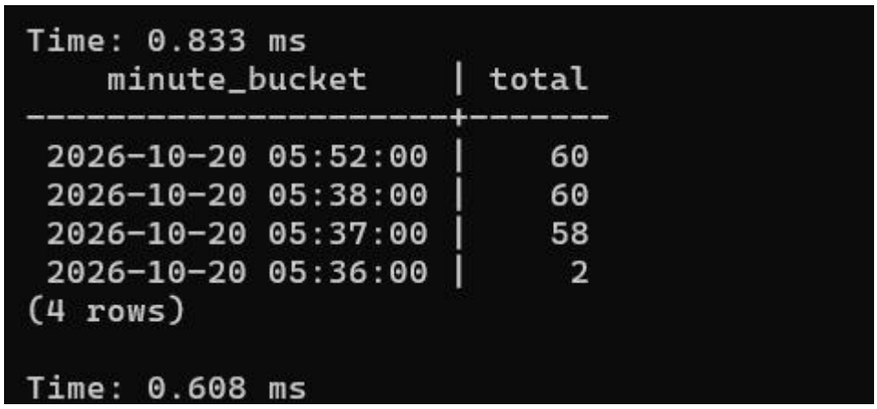
Time: 1.633 ms
```

```
Time: 1.633 ms
        time         |              tag               |                                log_message
---------------------+--------------------------------+---------------------------------------------------------------------------
 2026-10-20 05:52:30 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":20}
 2026-10-20 05:52:29 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":19}
 2026-10-20 05:52:28 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":18}
 2026-10-20 05:52:27 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":17}
 2026-10-20 05:52:26 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":16}
 2026-10-20 05:52:25 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":15}
 2026-10-20 05:52:24 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":14}
 2026-10-20 05:52:23 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":13}
 2026-10-20 05:52:22 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":12}
 2026-10-20 05:52:21 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":11}
 2026-10-20 05:52:20 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":10}
 2026-10-20 05:52:19 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":9}
 2026-10-20 05:52:18 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":8}
 2026-10-20 05:52:17 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":7}
 2026-10-20 05:52:16 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":6}
 2026-10-20 05:52:15 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":5}
 2026-10-20 05:52:14 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":4}
 2026-10-20 05:52:13 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":3}
 2026-10-20 05:52:12 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":2}
 2026-10-20 05:52:11 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":1}
 2026-10-20 05:38:36 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":20}
 2026-10-20 05:38:35 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":19}
 2026-10-20 05:38:34 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":18}
 2026-10-20 05:38:33 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":17}
 2026-10-20 05:38:32 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":16}
 2026-10-20 05:38:31 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":15}
 2026-10-20 05:38:30 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":14}
 2026-10-20 05:38:29 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":13}
 2026-10-20 05:38:28 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":12}
 2026-10-20 05:38:27 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":11}
 2026-10-20 05:38:26 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":10}
 2026-10-20 05:38:25 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":9}
 2026-10-20 05:38:24 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":8}
 2026-10-20 05:38:23 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":7}
 2026-10-20 05:38:22 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":6}
 2026-10-20 05:38:21 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":5}
 2026-10-20 05:38:20 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":4}
 2026-10-20 05:38:19 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":3}
 2026-10-20 05:38:18 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":2}
 2026-10-20 05:38:17 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":1}
 2026-10-20 05:37:18 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":20}
 2026-10-20 05:37:17 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":19}
 2026-10-20 05:37:16 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":18}
 2026-10-20 05:37:15 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":17}
 2026-10-20 05:37:14 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":16}
 2026-10-20 05:37:13 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":15}
 2026-10-20 05:37:12 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":14}
 2026-10-20 05:37:11 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":13}
 2026-10-20 05:37:10 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":12}
 2026-10-20 05:37:09 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":11}
 2026-10-20 05:37:08 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":10}
 2026-10-20 05:37:07 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":9}
 2026-10-20 05:37:06 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":8}
 2026-10-20 05:37:05 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":7}
 2026-10-20 05:37:04 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":6}
 2026-10-20 05:37:03 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":5}
 2026-10-20 05:37:02 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":4}
 2026-10-20 05:37:01 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":3}
 2026-10-20 05:37:00 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":2}
 2026-10-20 05:36:59 | docker.bab-6-generator-worker-1 | {"service":"worker","severity":"ERROR","event":"retry_test","attempt":1}
(60 rows)

Time: 0.833 ms
    minute_bucket    | total
---------------------+-------
 2026-10-20 05:52:00 |    60
```

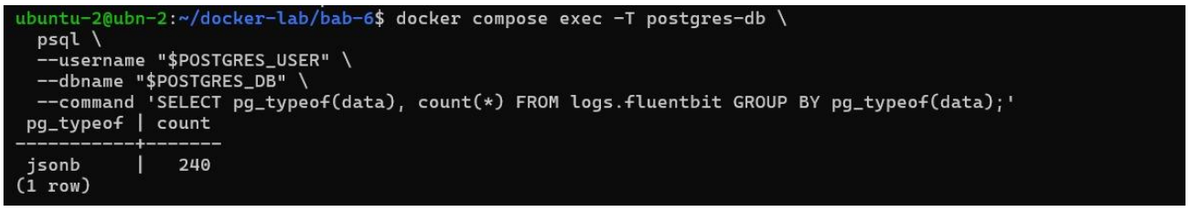
Kotak Perintah E — Memverifikasi Record PostgreSQL

Dilakukan pemeriksaan terhadap record yang berhasil disimpan di PostgreSQL. Query untuk menghitung jumlah record digunakan untuk mengetahui banyaknya log yang masuk, sedangkan query berdasarkan tag digunakan untuk melihat distribusi log dari masing-masing generator. Jika hasilnya menampilkan tag dari generator autentikasi, API, dan worker, berarti ketiga sumber telah mengirimkan log yang dapat ditemukan di database. Pemeriksaan juga dilakukan terhadap metadata seperti container_name, source, dan log yang disimpan dalam kolom JSONB. Metadata tersebut penting karena membantu mengidentifikasi asal log dan membaca isi pesan tanpa harus mengandalkan nama container saja.



Verifikasi tipe data dapat dilakukan dengan:

Dilakukan pemeriksaan terhadap struktur penyimpanan data log di PostgreSQL. Kolom data menggunakan tipe JSONB, yaitu tipe data PostgreSQL untuk menyimpan dokumen JSON dalam bentuk biner yang mendukung pemrosesan dan pengindeksan. Tipe ini memungkinkan informasi seperti nama container, sumber log, dan pesan diambil menggunakan operator JSON, misalnya data ->> 'source'. Pemeriksaan tipe data menggunakan pg_typeof(data) membantu memastikan bahwa kolom tersebut benar-benar bertipe jsonb, bukan sekadar teks biasa. Verifikasi ini penting karena query analisis pada praktikum bergantung pada kemampuan PostgreSQL mengakses properti JSON secara langsung.



6.1.5 Verifikasi PASS/FAIL

Verifikasi 1 — Validasi Compose

Verifikasi pertama dilakukan menggunakan perintah docker compose config --quiet. Perintah ini digunakan untuk memeriksa apakah konfigurasi Docker Compose dapat diproses tanpa kesalahan. Jika perintah selesai dengan kode keluar 0, konfigurasi dinyatakan valid dan dapat dilanjutkan ke tahap menjalankan layanan. Pemeriksaan ini penting karena kesalahan sintaks atau konfigurasi dapat menyebabkan layanan tidak berhasil dibuat.



Verifikasi 2 — Kesehatan Backend

Verifikasi kedua dilakukan dengan menjalankan docker compose ps untuk memeriksa status layanan PostgreSQL. Status healthy menunjukkan bahwa pemeriksaan kesehatan yang dikonfigurasi untuk PostgreSQL berhasil. Hal ini menjadi indikator bahwa backend database siap digunakan berdasarkan healthcheck yang ditetapkan. Pemeriksaan ini lebih informatif daripada hanya melihat apakah container berstatus berjalan, meskipun query database tetap diperlukan untuk membuktikan bahwa data dapat dibaca dan ditulis.



Verifikasi 3 — Kesehatan Collector

Verifikasi ketiga dilakukan dengan memeriksa endpoint kesehatan Fluent Bit melalui /api/v1/health. Respons HTTP dalam rentang 2xx menunjukkan bahwa permintaan berhasil diproses. Pemeriksaan ini digunakan untuk memastikan bahwa endpoint monitoring dapat diakses dan memberikan respons sukses. Namun, hasil tersebut tidak secara langsung membuktikan bahwa seluruh log telah masuk ke PostgreSQL, sehingga perlu dikombinasikan dengan pemeriksaan record database.



Verifikasi 4 — Tiga Sumber Log

Verifikasi keempat dilakukan menggunakan query GROUP BY tag untuk mengelompokkan record berdasarkan tag sumber log. Hasil yang menampilkan sedikitnya tiga tag berbeda menjadi indikator bahwa log dari tiga generator, yaitu autentikasi, API, dan worker, telah diterima. Pemeriksaan ini penting karena keberhasilan pipeline tidak hanya ditentukan oleh adanya data, tetapi juga oleh kemampuan sistem menerima log dari beberapa sumber yang berbeda.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ set -a
source .env
set +a

docker compose exec -T postgres-db \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT tag, COUNT(*) AS jumlah_event
      FROM logs.fluentbit
      GROUP BY tag
      ORDER BY tag;"
              tag               | jumlah_event
--------------------------------+--------------
 docker.bab-6-generator-api-1    |           80
 docker.bab-6-generator-auth-1   |           80
 docker.bab-6-generator-worker-1 |           80
(3 rows)

ubuntu-2@ubn-2:~/docker-lab/bab-6$
```

Verifikasi 5 — Struktur Record

Verifikasi kelima dilakukan untuk memastikan bahwa record log memiliki field penting, yaitu container_name, source, dan log. Field container_name digunakan untuk mengidentifikasi nama container pengirim, source menunjukkan asal log, dan log berisi pesan yang dicatat. Kelengkapan field tersebut membantu proses penelusuran sumber masalah dan analisis log. Dengan demikian, data yang tersimpan tidak hanya berupa kumpulan pesan, tetapi juga memiliki informasi pendukung yang berguna.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ docker compose exec -T postgres-db \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT tag, jsonb_pretty(data)
      FROM logs.fluentbit
      ORDER BY time DESC
      LIMIT 3;"
              tag               |                                                
```

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ docker compose exec -T postgres-db \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT column_name, data_type, udt_name
      FROM information_schema.columns
      WHERE table_schema = 'logs'
        AND table_name = 'fluentbit'
        AND column_name = 'data';"
 column_name | data_type | udt_name
-------------+-----------+----------
 data        | jsonb     | jsonb
(1 row)

ubuntu-2@ubn-2:~/docker-lab/bab-6$ docker compose exec -T postgres-db \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT pg_typeof(data) AS tipe_data, COUNT(*) AS jumlah
      FROM logs.fluentbit
      GROUP BY pg_typeof(data);"
 tipe_data | jumlah
-----------+--------
 jsonb     |    240
(1 row)
```

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ chmod +x scripts/test-buffering.sh
./scripts/test-buffering.sh
[INFO] Jumlah awal: 240
[+] stop 1/1
 ✔Container bab-6-postgres-db-1 Stopped
[+] up 5/5
 ✔Container bab-6-fluent-bit-1       Running
 ✔Container bab-6-generator-api-1    Started
 ✔Container bab-6-generator-auth-1   Started
 ✔Container bab-6-generator-worker-1 Started
 ✔Container bab-6-postgres-db-1      Healthy
[INFO] PostgreSQL berhenti; generator berjalan selama 25 detik.
[PASS] Jumlah sesudah pemulihan: 300
[INFO] Kenaikan record: 60
```

Verifikasi 7 — Pemulihan Output

Verifikasi ketujuh dilakukan menggunakan test-buffering.sh untuk memeriksa apakah jumlah record bertambah setelah PostgreSQL kembali aktif. Script membandingkan jumlah record sebelum dan sesudah pengujian. Jika jumlah record meningkat, script menampilkan [PASS] beserta jumlah penambahan record. Hasil ini menunjukkan bahwa terdapat log baru yang berhasil tersimpan setelah database pulih dari gangguan sementara. Pengujian tersebut mendukung penilaian bahwa pipeline dapat melanjutkan pengiriman log setelah database kembali tersedia. Namun, untuk memastikan tidak ada log yang hilang, diperlukan pemeriksaan tambahan terhadap jumlah log yang dihasilkan dan jumlah log yang berhasil diterima.

```
ubuntu-2@ubn-2:~/docker-lab/bab-6$ cat scripts/test-buffering.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

count_rows() {
    docker compose exec -T postgres-db \
        psql \
        --username "$POSTGRES_USER" \
        --dbname "$POSTGRES_DB" \
        --tuples-only \
        --no-align \
        --command 'SELECT count(*) FROM logs.fluentbit;'
}

before="$(count_rows)"
echo "[INFO] Jumlah awal: $before"

docker compose stop postgres-db

docker compose up -d --force-recreate \
    generator-auth \
    generator-api \
    generator-worker

echo "[INFO] PostgreSQL berhenti; generator berjalan selama 25 detik."
sleep 25

docker compose start postgres-db

for attempt in {1..30}; do
    if docker compose exec -T postgres-db \
        pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" >/dev/null 2>&1; then
        break
    fi

    if [[ "$attempt" -eq 30 ]]; then
        echo "[FAIL] PostgreSQL tidak kembali sehat." >&2
        exit 1
    fi
    sleep 2
done

for attempt in {1..60}; do
    after="$(count_rows)"
    if (( after > before )); then
        echo "[PASS] Jumlah sesudah pemulihan: $after"
        echo "[INFO] Kenaikan record: $((after - before))"
        exit 0
    fi
    sleep 2
done

echo "[FAIL] Tidak ada record baru setelah PostgreSQL pulih." >&2
echo "[INFO] Periksa: docker compose logs --tail 200 fluent-bit" >&2
exit 1
```

Verifikasi 8 — Pembatasan Jaringan PostgreSQL

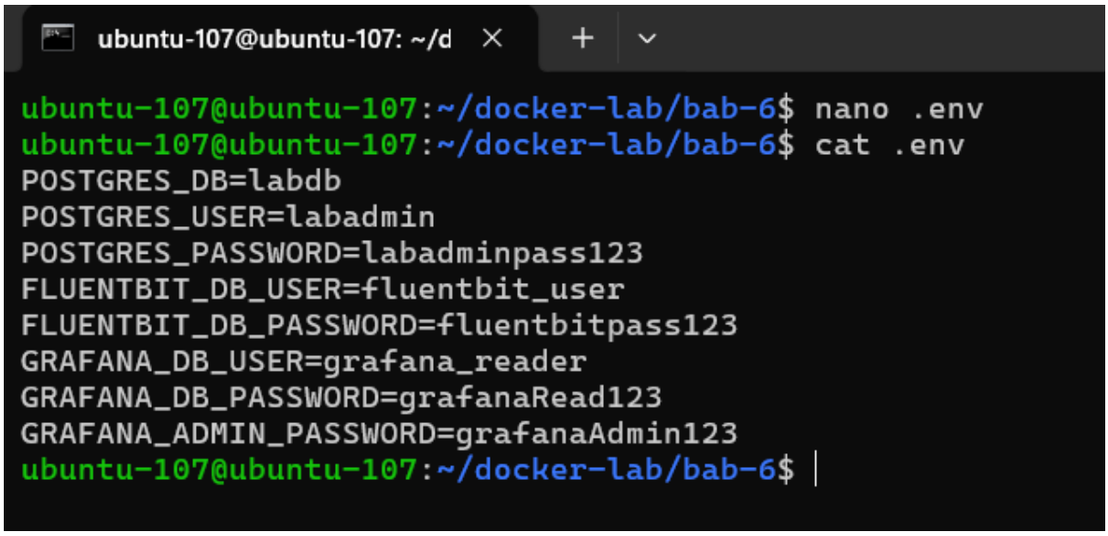
Verifikasi kedelapan dilakukan untuk memastikan bahwa PostgreSQL tidak memiliki published port ke host. PostgreSQL tetap dapat diakses oleh Fluent Bit melalui jaringan internal Docker Compose menggunakan nama layanan dan port database 5432. Tidak memublikasikan port database membantu mengurangi risiko akses langsung dari luar lingkungan container. Pada konfigurasi ini, port monitoring dan penerimaan log Fluent Bit dibatasi ke alamat 127.0.0.1, sehingga tidak langsung dibuka ke seluruh jaringan. Pembatasan tersebut merupakan langkah keamanan untuk mengurangi permukaan serangan dan memastikan layanan hanya membuka akses yang dibutuhkan.



Penambahan Grafana diluar Cheatsheet BAB 6

Konfigurasi Kredensial dan Penyiapan Hak Akses Basis Data

Pada tahap awal, konfigurasi variabel lingkungan disimpan di dalam berkas .env untuk memusatkan parameter sensitif seperti nama basis data (POSTGRES_DB=labdb), akun administrator PostgreSQL, serta kredensial untuk Fluent Bit dan Grafana (GRAFANA_DB_USER=grafana_reader, GRAFANA_DB_PASSWORD, dan GRAFANA_ADMIN_PASSWORD). Selanjutnya, pembuatan peran (*role*) baru bernama grafana_reader dilakukan langsung dari container PostgreSQL menggunakan perintah docker compose exec. Akun ini dikonfigurasi berdasarkan prinsip *least privilege*, di mana hak akses yang diberikan hanya sebatas CONNECT ke database labdb, USAGE pada skema logs, serta hak SELECT untuk seluruh tabel dalam skema



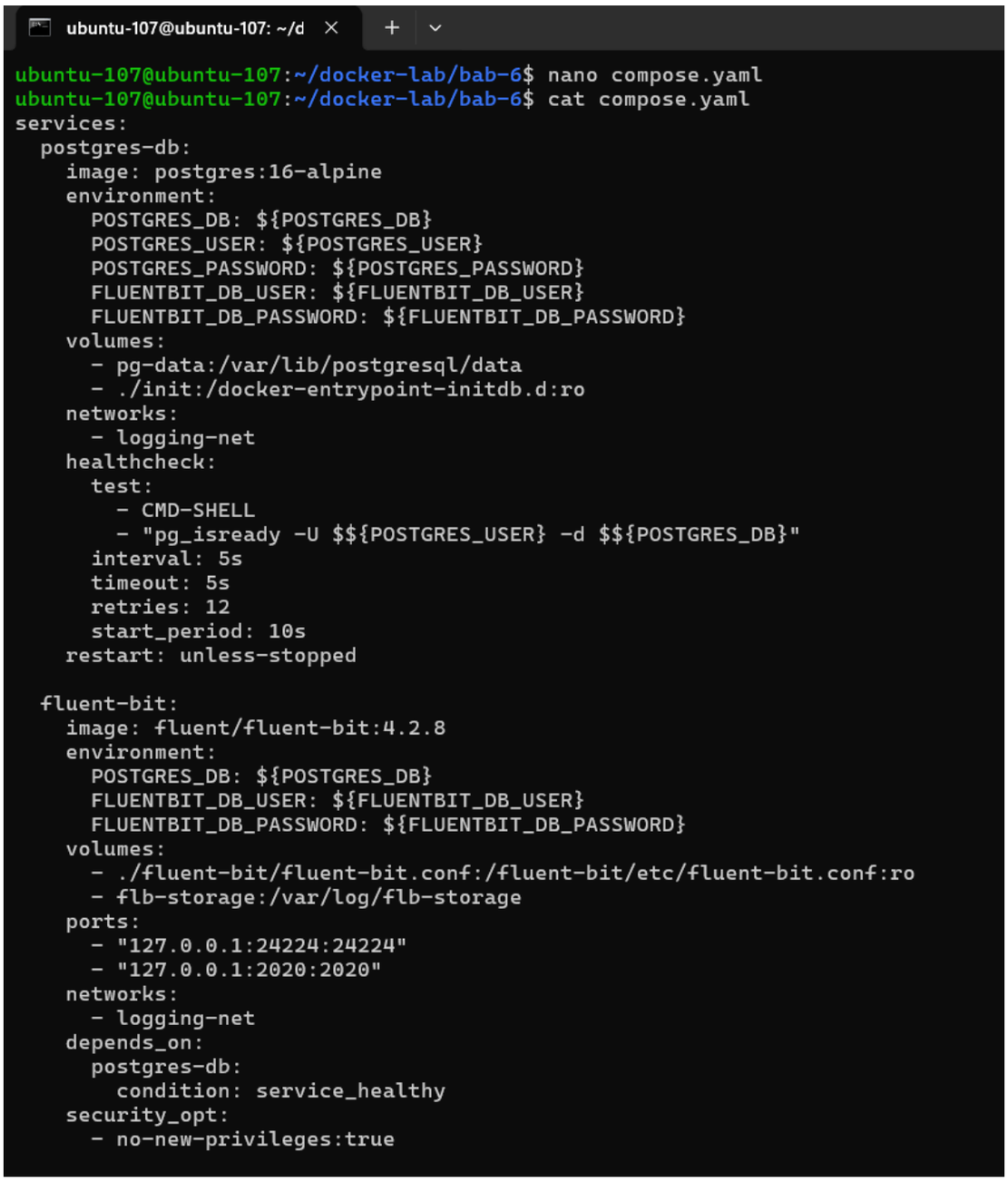
Pengaturan Service pada Docker Compose dan Penyebaran Stack

Definisi infrastruktur disusun di dalam berkas compose.yaml yang mencakup service postgres-db berbasis *image* postgres:16-alpine lengkap dengan pengujian kesehatan (*healthcheck*) pg_isready. Service fluent-bit dikonfigurasi untuk mengekspos port 24224 untuk penerimaan log Forward dan port 2020 untuk pemantauan HTTP. Penambahan service grafana menggunakan *image* grafana/grafana:latest dilakukan dengan mengaitkan kata sandi admin dari file lingkungan, mempublikasikan port 3000, serta menetapkan dependensi agar Grafana baru berjalan setelah postgres-db dalam kondisi sehat. Setelah berkas dipastikan valid menggunakan docker compose config quiet, seluruh container dieksekusi dengan docker compose up -d. Hasil verifikasi melalui docker compose ps mengonfirmasi bahwa seluruh service utama (fluent-bit, grafana, dan postgres-db) telah berstatus aktif dan sehat.



Pengalihan Port SSH dan Akses Antarmuka Grafana

Proses inisialisasi awal sistem dibuktikan dengan berjalannya skrip migrasi basis data secara otomatis untuk membuat tabel pendukung dan indeks. Untuk mengakses antarmuka Grafana dari mesin lokal secara aman, dijalankan perintah pengalihan port SSH (*Local Port Forwarding*) melalui perintah ssh -L 3000:127.0.0.1:3000 ubuntu-2@10.252.108.103. Setelah pengalihan port aktif, antarmuka web Grafana diakses melalui URL http://localhost:3000 yang menampilkan halaman *login* resmi Grafana v12.2.4. Pengguna kemudian melakukan autentikasi hingga berhasil masuk ke halaman utama *Welcome to Grafana*.



```
ubuntu-107@ubuntu-107: ~/d
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD}
    ports:
      - "127.0.0.1:3000:3000"
    networks:
      - logging-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped

  generator-auth:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"auth","severity":"INFO","event":"login_test","sequence":%s}\n' "$$i"
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit

  generator-api:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"api","severity":"WARN","event":"latency_test","duration_ms":%s}\n' "$$((i * 10))"
```

3

```
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit

  generator-worker:
    image: alpine:3.20
    command:
      - /bin/sh
      - -c
      - |
        i=1
        while [ "$$i" -le 20 ]; do
          printf '{"service":"worker","severity":"ERROR","event":"retry_test","attempt":%s}\n' "$$i"
          i=$$((i + 1))
          sleep 1
        done
    logging:
      driver: fluentd
      options:
        fluentd-address: "127.0.0.1:24224"
        fluentd-async: "true"
        tag: "docker.{{.Name}}"
    depends_on:
      - fluent-bit

volumes:
  pg-data:
  flb-storage:
  grafana-storage:

networks:
  logging-net:
    driver: bridge
ubuntu-107@ubuntu-107:~/docker-lab/bab-6$
```

Konfigurasi Data Source PostgreSQL

Integrasi basis data dilakukan melalui menu *Data Sources* di Grafana dengan menambahkan koneksi baru berlabel grafana-postgresql-datasource. Parameter utama diisi dengan mengarahkan *Host* ke postgres-db:5432, nama basis data labdb, akun pengguna grafana_reader, serta mengonfigurasi opsi SSL ke posisi *disable*. Pengujian koneksi berhasil divalidasi dengan munculnya pesan *Database Connection OK*. Selanjutnya, pengujian kueri SQL pada menu *Explore* membuktikan bahwa Grafana mampu membaca log terstruktur dari tabel logs.fluentbit secara langsung.

```
ubuntu-107@ubuntu-107: ~/d
ubuntu-107@ubuntu-107:~/docker-lab/bab-6$ docker compose config --quiet
ubuntu-107@ubuntu-107:~/docker-lab/bab-6$ docker compose up -d grafana
[+] up 13/13
 ✔Image grafana/grafana:latest  Pulled                                                                                    104.7s
 ✔Container bab-6-postgres-db-1 Healthy                                                                                     1.7s
 ✔Container bab-6-grafana-1     Started                                                                                     2.4s
ubuntu-107@ubuntu-107:~/docker-lab/bab-6$ docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE       CREATED          STATUS                    PORTS
bab-6-fluent-bit-1  fluent/fluent-bit:4.2.2        "/fluent-bit/bin/flu…"   fluent-bit    18 minutes ago   Up 18 minutes             127.0.0.1:2020->2020/tcp, 127.0.0.1:2422
4->24224/tcp
bab-6-grafana-1     grafana/grafana:latest         "/run.sh"                grafana       7 seconds ago    Up 5 seconds              127.0.0.1:3000->3000/tcp
bab-6-postgres-db-1 postgres:16-alpine             "docker-entrypoint.s…"   postgres-db   18 minutes ago   Up 11 minutes (healthy)   5432/tcp
ubuntu-107@ubuntu-107:~/docker-lab/bab-6$
```

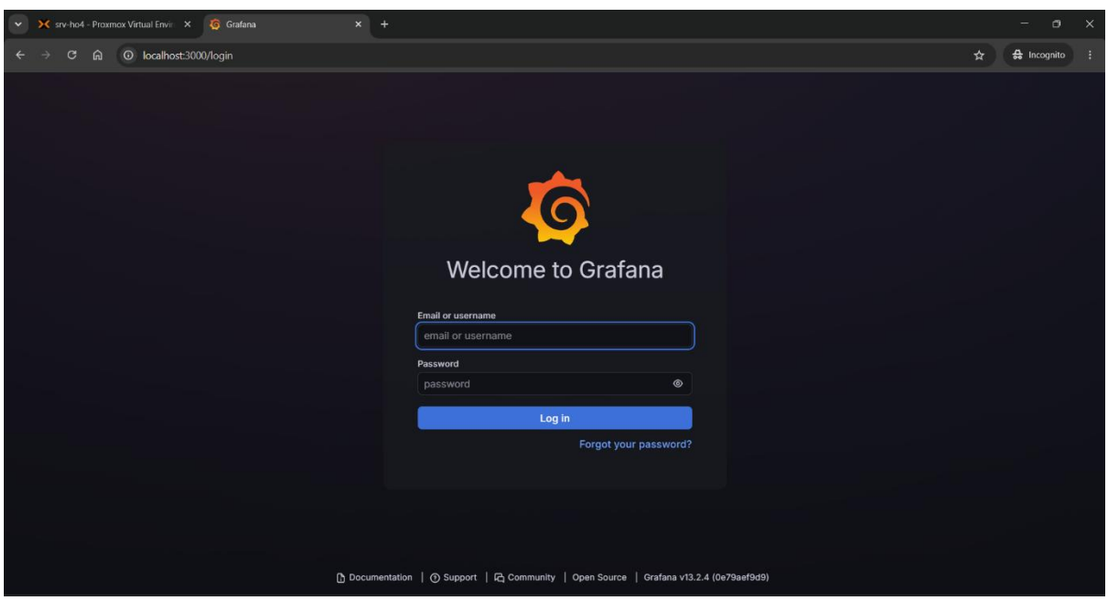
Visualisasi dan Pembuatan Dashboard Log

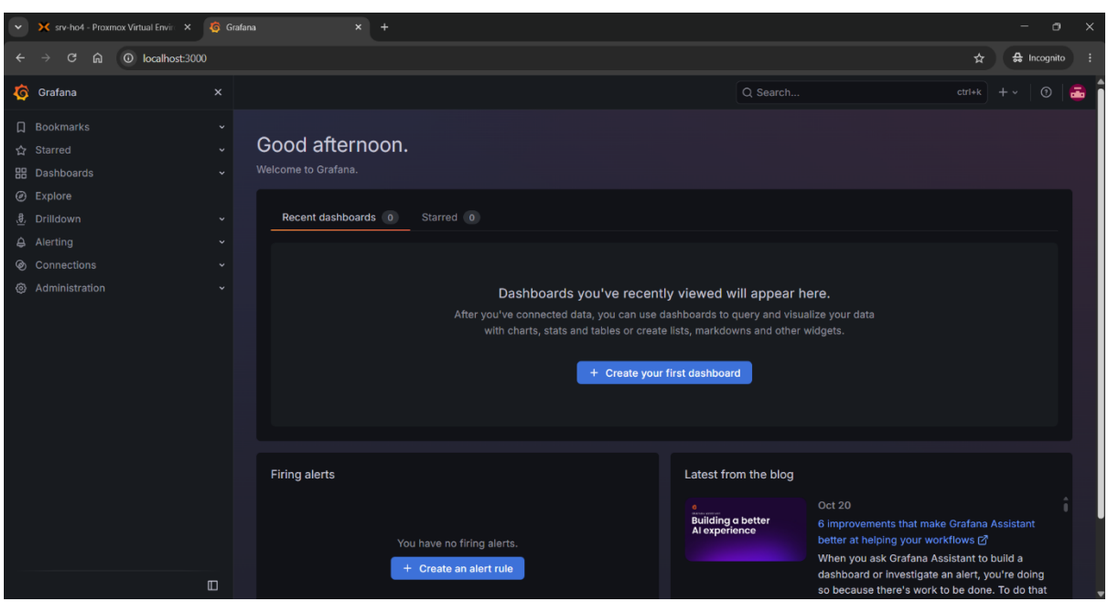
Tahap akhir praktikum difokuskan pada penyusunan *dashboard* visualisasi data log. Panel pertama dikonfigurasi dalam bentuk tampilan tabel (*Table View*) yang menyajikan rincian entri log secara terstruktur, mencakup stempel waktu (*timestamp*), tag generator container, dan pesan log format JSON. Selain itu, ditambahkan panel grafik tren (*time-series chart*) untuk memvisualisasikan volume dan sebaran kedatangan log dari berbagai service generator berdasarkan rentang waktu secara *real-time*.

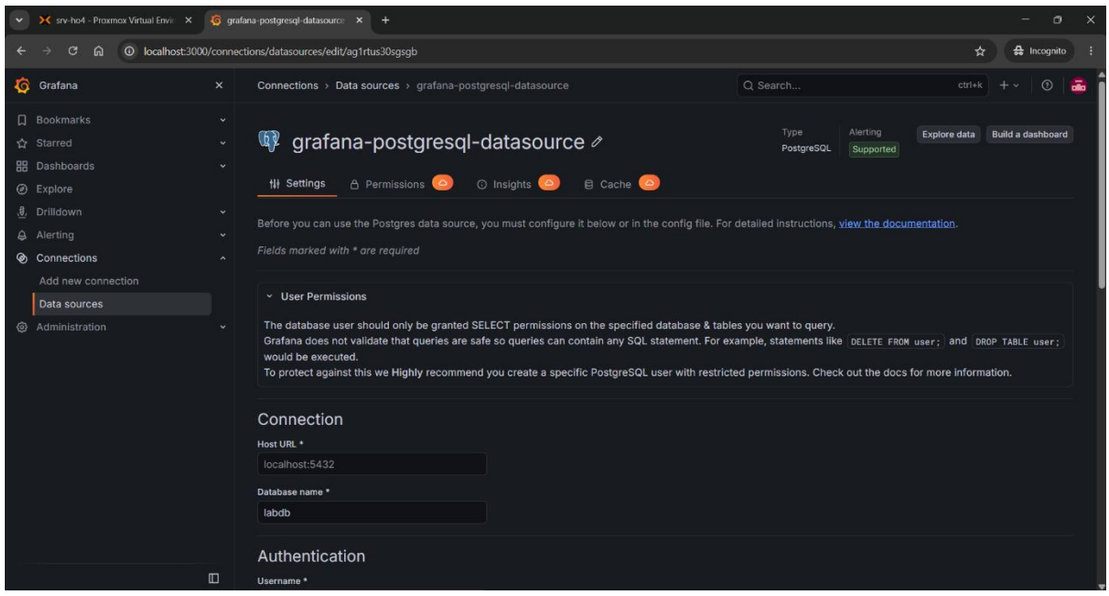


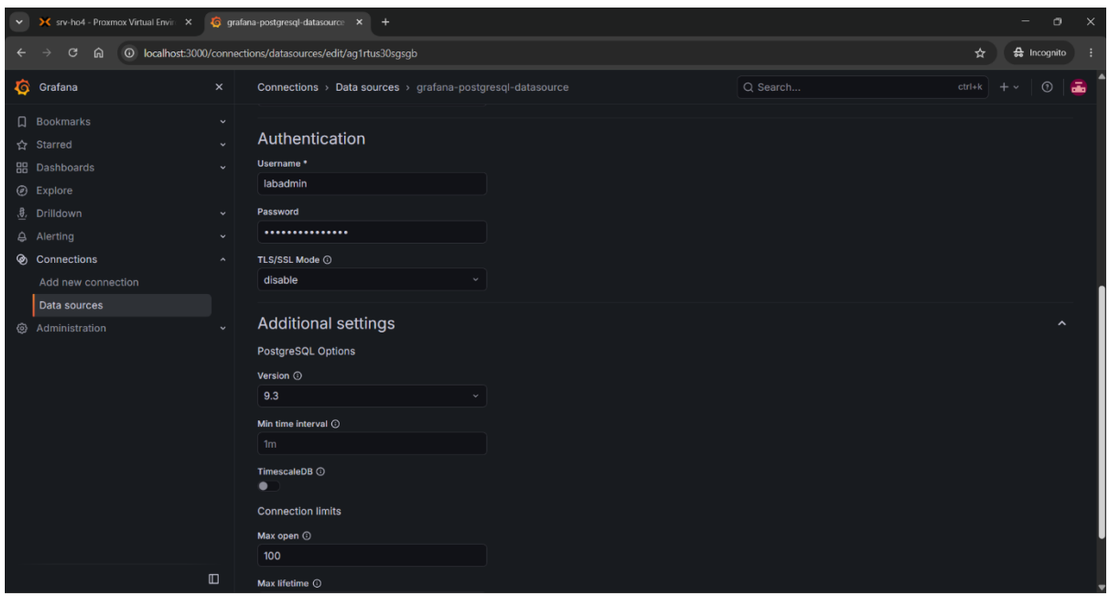
ssh -L 3000:127.0.0.1:3000 ubuntu-2@10.252.108.103

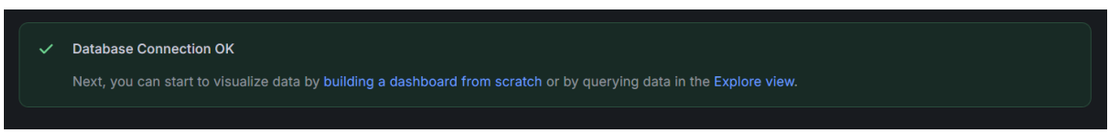
http://localhost:3000

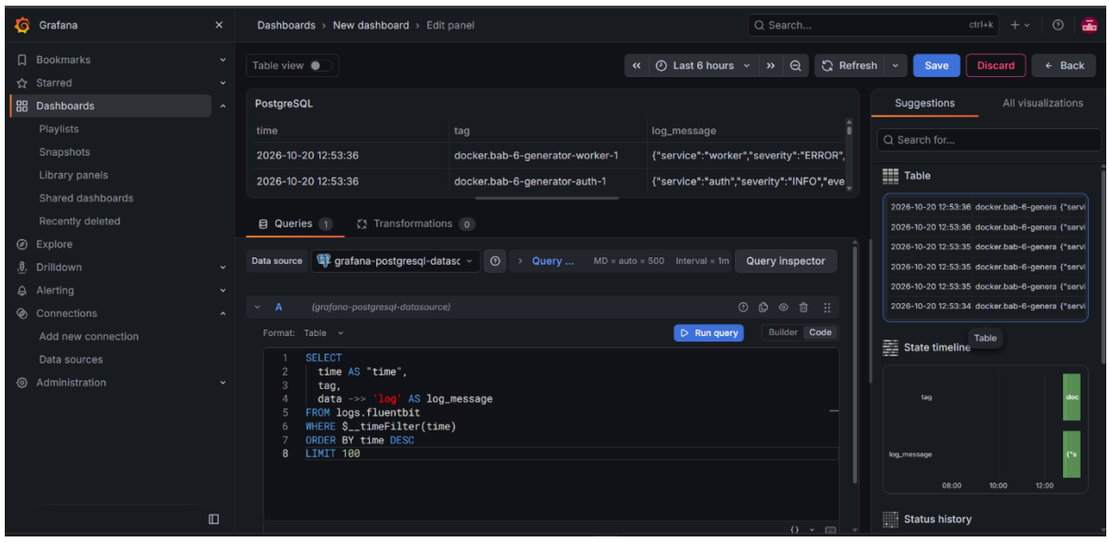


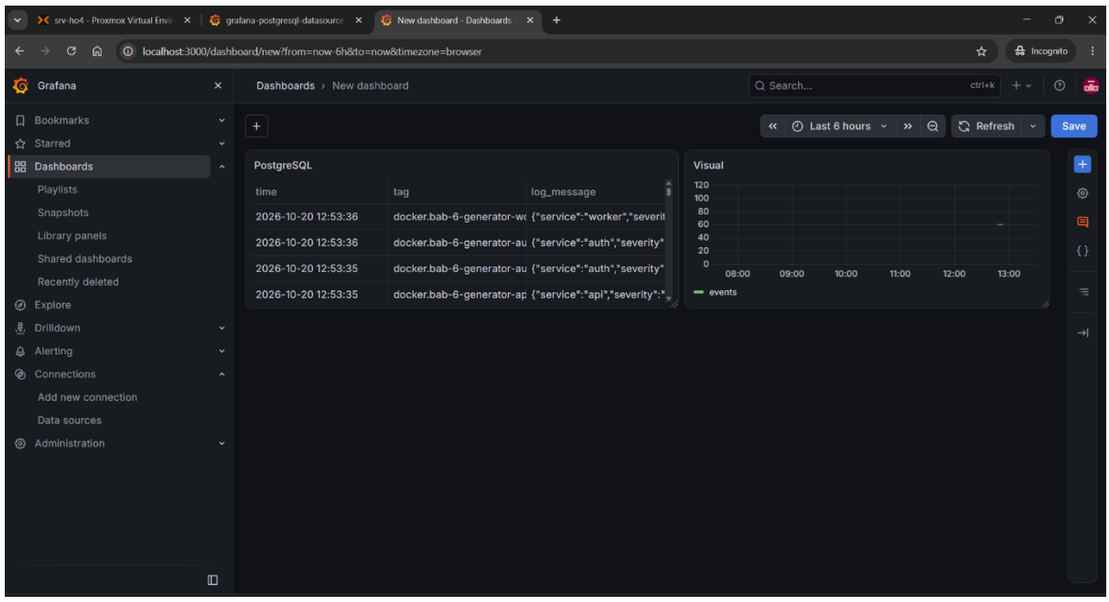



srv-ho4 - Proxmox Virtual Envi
grafana-postgresql-datasource
localhost:3000/connections/datasources/edit/ag1rtus30sgsgb
Incognito
Grafana
Connections › Data sources › grafana-postgresql-datasource
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
grafana-postgresql-datasource
Type
PostgreSQL
Alerting
Supported
Explore data
Build a dashboard
Settings
Permissions
Insights
Cache
Before you can use the Postgres data source, you must configure it below or in the config file. For detailed instructions, view the documentation.
Fields marked with * are required
User Permissions
The database user should only be granted SELECT permissions on the specified database & tables you want to query.
Grafana does not validate that queries are safe so queries can contain any SQL statement. For example, statements like DELETE FROM user; and DROP TABLE user; would be executed.
To protect against this we Highly recommend you create a specific PostgreSQL user with restricted permissions. Check out the docs for more information.
Connection
Host URL *
localhost:5432
Database name *
labdb
Authentication
Username *


srv-ho4 - Proxmox Virtual Envi
grafana-postgresql-datasource
localhost:3000/connections/datasources/edit/ag1rtus30sgsgb
Incognito
Grafana
Connections › Data sources › grafana-postgresql-datasource
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Authentication
Username *
labadmin
Password
TLS/SSL Mode
disable
Additional settings
PostgreSQL Options
Version
9.3
Min time interval
1m
TimescaleDB
Connection limits
Max open
100
Max lifetime


Database Connection OK
Next, you can start to visualize data by building a dashboard from scratch or by querying data in the Explore view.


Grafana
Dashboards › New dashboard › Edit panel
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Table view
Last 6 hours
Refresh
Save
Discard
Back
PostgreSQL
time
tag
log_message
2026-10-20 12:53:36
docker.bab-6-generator-worker-1
{"service":"worker","severity":"ERROR",
2026-10-20 12:53:36
docker.bab-6-generator-auth-1
{"service":"auth","severity":"INFO","eve
Queries 1
Transformations 0
Data source
grafana-postgresql-datasc
Query ...
MD = auto = 500
Interval = 1m
Query inspector
A
(grafana-postgresql-datasource)
Format: Table
Run query
Builder
Code
SELECT
time AS "time",
tag,
data ->> 'log' AS log_message
FROM logs.fluentbit
WHERE $__timeFilter(time)
ORDER BY time DESC
LIMIT 100
Suggestions
All visualizations
Search for...
Table
State timeline
Status history
08:00
10:00
12:00


srv-ho4 - Proxmox Virtual Envi
grafana-postgresql-datasource
New dashboard - Dashboards
localhost:3000/dashboard/new?from=now-6h&to=now&timezone=browser
Incognito
Grafana
Dashboards › New dashboard
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Last 6 hours
Refresh
Save
PostgreSQL
time
tag
log_message
2026-10-20 12:53:36
docker.bab-6-generator-wc
{"service":"worker","severit
2026-10-20 12:53:36
docker.bab-6-generator-au
{"service":"auth","severity"
2026-10-20 12:53:35
docker.bab-6-generator-au
{"service":"auth","severity"
2026-10-20 12:53:35
docker.bab-6-generator-ap
{"service":"api","severity":"
Visual
120
100
80
60
40
20
0
08:00
09:00
10:00
11:00
12:00
13:00
events